

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: May 21, 2026

Lab # 14

Environment Setup:

Navigated from the home directory to the **Documents** folder using the **cd** command and initialized a structured workspace by creating a new directory named **Hash_Lab** with the **mkdir** command. The final command executes **cd Hash_Lab**, successfully shifting the active terminal shell location into the newly created directory to begin the practical lab tasks.

```
(kali㉿kali)-[~]  
$ cd Documents  
  
(kali㉿kali)-[~/Documents]  
$ mkdir Hash_Lab  
  
(kali㉿kali)-[~/Documents]  
$ cd Hash_Lab
```

1. Create a plaintext file

The **nano** text editor was used to create a file named **message.txt**. The contents were verified using the **cat** command, confirming the exact input string: *Today is Thursday. We are doing Hash Lab.*

```
(kali㉿kali)-[~/Documents/Hash_Lab]  
$ nano message.txt  
  
(kali㉿kali)-[~/Documents/Hash_Lab]  
$ cat message.txt  
Today is Thursday. We are doing Hash Lab.
```

2. Generate MD5 Hash

The OpenSSL utility executes the legacy MD5 algorithm via **openssl dgst -md5** to process the file's contents, generating a fixed-length 128-bit (32 hex characters) cryptographic checksum.

```
(kali㉿kali)-[~/Documents/Hash_Lab]  
$ openssl dgst -md5 message.txt  
MD5(message.txt)= 913f9e9d5f34acf887d59036b64e423f
```

3. Generate SHA-256 Hash

Increased security strength by using the **-sha256** flag, producing a unique, industry-standard 256-bit (64 hex characters) message digest of the same text file.

```
(kali㉿kali)-[~/Documents/Hash_Lab]  
$ openssl dgst -sha256 message.txt  
SHA2-256(message.txt)= 8ed3b4eddae701656b08725df39c9c5776aee77da2b375a41b05aaefca6ff0c
```

4. Generate SHA-512 Hash

The command computes a highly collision-resistant 512-bit (128 hex characters) digest using SHA-512, showcasing how a longer hash size dramatically minimizes the probability of duplicate outputs.

```
(kali㉿kali)-[~/Documents/Hash_Lab]  
$ openssl dgst -sha512 message.txt  
SHA2-512(message.txt)= 31060841c8c2110a37f6f501e8fb9c8c39f4e877bbb1a62b3f3930470058a36e4320d265c15bcfc0a0948d52bc609f848e93c1341659395fc9bec94ff1baed38
```

5. Store the Hash value in a file

Leveraged Linux I/O redirection operators to aggregate multiple cryptographic results, using `>` to write the baseline MD5 digest into a new file named **hashes.txt**, followed by `>>` to append the **SHA-256** and **SHA-512** digests seamlessly to the end of the same file. Executing **cat hashes.txt** displays the consolidated records inside the file, validating that all three unique cryptographic checksums were successfully captured and saved sequentially without overwriting previous data.

```
(kali㉿kali)-[~/Documents/Hash_Lab]
$ openssl dgst -md5 message.txt > hashes.txt

(kali㉿kali)-[~/Documents/Hash_Lab]
$ openssl dgst -sha256 message.txt >> hashes.txt

(kali㉿kali)-[~/Documents/Hash_Lab]
$ openssl dgst -sha512 message.txt >> hashes.txt
```

```
(kali㉿kali)-[~/Documents/Hash_Lab]
$ cat hashes.txt
MD5(message.txt)= 913f9e9d5f34acf887d59036b64e423f
SHA2-256(message.txt)= 8ed3b4eddae701656b08725df39c9c5776aee77da2b375a41b05aaefca6ff0c
SHA2-512(message.txt)= 31060841c8c2110a37f6f501e8fb9c8c39f4e877bbb1a62b3f3930470058a36e4320d265c15bcfc0a0948d52bc609f848e93c1341659395fc9bec94ff1baed38
```

6. Modify the File and Recalculate Hash

Appends a new string to **message.txt** using `>>` and recalculates its **MD5**, **SHA-256**, and **SHA-512** digests, outputting them into a new file called **hashes_modified.txt**. A one-line chained command prints both the original and modified hashes sequentially, demonstrating the avalanche effect where a minor change in input data completely alters every cryptographic checksum.

```
(kali㉿kali)-[~/Documents/Hash_Lab]
$ echo "Modified Content: This is new text being added." >> message.txt

(kali㉿kali)-[~/Documents/Hash_Lab]
$ cat message.txt
Today is Thursday. We are doing Hash Lab.
Modified Content: This is new text being added.
```

```
(kali㉿kali)-[~/Documents/Hash_Lab]
$ openssl dgst -md5 message.txt > hashes_modified.txt

(kali㉿kali)-[~/Documents/Hash_Lab]
$ openssl dgst -sha256 message.txt >> hashes_modified.txt

(kali㉿kali)-[~/Documents/Hash_Lab]
$ openssl dgst -sha512 message.txt >> hashes_modified.txt
```

```
(kali㉿kali)-[~/Documents/Hash_Lab]
$ echo "=== ORIGINAL HASHES ===" && cat hashes.txt && echo "" && echo "=== MODIFIED HASHES ===" && cat hashes_modified.txt
=== ORIGINAL HASHES ===
MD5(message.txt)= 913f9e9d5f34acf887d59036b64e423f
SHA2-256(message.txt)= 8ed3b4eddae701656b08725df39c9c5776aee77da2b375a41b05aaefca6ff0c
SHA2-512(message.txt)= 31060841c8c2110a37f6f501e8fb9c8c39f4e877bbb1a62b3f3930470058a36e4320d265c15bcfc0a0948d52bc609f848e93c1341659395fc9bec94ff1baed38

=== MODIFIED HASHES ===
MD5(message.txt)= 260dbe74fd33567e6525891e79ccc91c
SHA2-256(message.txt)= 037e463321b426be9aa3328aa24053f3c588ff04b15b0d34811dcc70af398eca
SHA2-512(message.txt)= ddbd8563612a4f455069fff7e04f5e6126fbcf189007c1fea79c6516eac12d8a0ad477937729f98398ac2aa6e8c2d5b850e182c23815f5a6e49f9208b620c776
```

7. Observation/Output

Algor ithm	Original Hash	Modified Hash
MD5	913f9e9d5f34acf887d59036b64e423f	260dbe74fd33567e6525891e79ccc91c
SHA-256	8ed3b4eddae701656b08725df39c9c5776aee77da2b375a41b05aaefca6ff0c	037e463321b426be9aa3328aa24053f3c588ff04b15b0d34811dcc70af398eca
SHA-512	31060841c8c2110a37f6f501e8fb9c8c39f4e877b bb1a62b3f3930470058a36e4320d265c15bcfc0a0 948d52bc609f848e93c1341659395fc9bec94ff1b aed38	ddbd8563612a4f455069fff7e04f5e6126fbcf189007 c1fea79c6516eac12d8a0ad477937729f98398ac2aa 6e8c2d5b850e182c23815f5a6e49f9208b620c776

Even a minor change to the input file (adding one line) produced in completely different hash values across all three algorithms. The original and altered hashes do not exhibit any apparent similarity.

8. Command Explanation

Command	Explanation
echo "... " > message.txt	Creates a plain text file with the given content
openssl dgst -md5 message.txt	Generates the MD5 digest (128-bit hash) of the file
openssl dgst -sha256 message.txt	Generates the SHA-256 digest (256-bit hash)
openssl dgst -sha512 message.txt	Generates the SHA-512 digest (512-bit hash)
>> hashes.txt	Appends output to a file without overwriting
echo "... " >> message.txt	Appends new content to the existing file

9. Result

All three hash algorithms successfully generated unique fixed-length digests for the input file. Upon modifying the file, all hash values changed completely, demonstrating the avalanche effect. The hashes were stored and verified using OpenSSL command-line tools on Kali Linux.

10. Conclusion

This lab demonstrated the practical use of cryptographic hash functions using OpenSSL. MD5, SHA-256, and SHA-512 each produced distinct hash values for the same file. The experiment confirmed that hash functions are sensitive to any change in input — even appending a single line produced an entirely different digest. SHA-256 and SHA-512 are preferred in modern security applications due to known weaknesses in MD5. Hash functions are fundamental to data integrity verification, digital signatures, and password storage.